龙芯 2K300/301 系列板卡使用手册-ncnn 神经网络推理

龙芯 2K300/301 开源库使用说明

ncnn 神经网络推理篇

一、ncnn 模块简介

ncnn 是腾讯优图实验室开源的高性能神经网络前向计算框架，专为移动端与嵌入式设备优化，无第三方依赖、跨平台兼容性强，针对 CPU 做了极致优化，是龙芯 2K300/301 板卡部署 AI 模型的首选推理框架。

龙芯 2K300/301 开源库提供了 NCNN 封装类 LQ_NCNN，底层已完成 LoongArch 架构适配，开发者无需关注底层优化细节，直接调用封装接口即可完成模型加载、图像预处理、推理预测等操作，大幅简化嵌入式 AI 开发流程。

1.1 核心特性

- 无第三方依赖，跨平台支持，适配龙芯 2K300/301 LoongArch 架构
- 支持 CPU 加速，可借助龙芯板卡算力实现高效推理
- 内存占用低，运行速度快，适配嵌入式设备有限资源
- 支持多种常见网络结构，兼容主流模型格式
- 提供 LQ_NCNN 封装类，支持模型加载、推理、自动图像预处理等功能
- 支持多任务场景，可与摄像头、UDP 通信等模块协同使用

1.2 适用场景

- 图像分类：通过预训练模型实现物体类别识别，如动物、植物、日常物品分类
- 目标检测：识别图像中的目标位置与类别，适用于智能车避障、监控检测等场景
- 图像分割：对图像进行像素级分类，适用于工业缺陷检测、场景分割等
- 人脸识别：基于人脸检测与特征提取，实现身份识别、人脸验证等功能

- 实时推理：结合摄像头实现实时图像分析，适配嵌入式实时应用需求

重要说明：LQ_NCNN 类目前完全适配 LQ_TinyClassifier 模型，可直接使用；如需使用其他模型（如 MobileNet、ResNet 等），需要自行编写推理代码，可参考 LQ_NCNN 类的实现进行适配。

【LQ_TinyClassifier 模型训练资料】

链接: https://pan.baidu.com/s/1Yz7P73Ag9T31zfgbtxE6cA

提取码: tihd

二、ncnn 库文件位置

龙芯 2K300/301 开源库中，ncnn 相关文件按固定目录结构存放，包含头文件、实现文件、静态库及工具链，便于开发者调用与调试。

2.1 头文件和实现文件

- 头文件：libraries/drv/inc/lq_ncnn.hpp
- 实现文件：libraries/drv/src/lq_ncnn.cpp
- 静态库文件：libraries/drv/src/liblqncnn.a

2.2 ncnn 工具链

工具链位置：tools/LQ_Dep_libs/ncnn_install/

包含工具明细：

```plain text
tools/LQ_Dep_libs/ncnn_install/
├── bin/
│   ├── ncnn2int8      # 模型量化工具（将模型转换为 INT8 精度，提升推理速度）
│   ├── ncnn2mem       # 模型内存格式转换工具
│   ├── ncnn2table     # 量化表生成工具
│   ├── ncnnoptimize   # 模型优化工具（优化模型结构，提升推理效率）
│   └── ncnnmerge      # 模型合并工具
└── lib/
    └── libncnn.a      # NCNN 静态库（底层推理核心）
```

三、LQ_NCNN 类接口说明

LQ_NCNN 类是龙芯开源库对 ncnn 原生 API 的封装，提供简洁统一的接口，支持模型加载、参数配置、图像推理等核心功能，调用便捷、稳定性高。

3.1 构造与析构

【构造函数】

```
c++
LQ_NCNN()
```

功能：创建 NCNN 对象，使用默认参数初始化，无需额外配置。

示例：

```
c++
// 创建 LQ_NCNN 对象
LQ_NCNN ncnn;
```

【析构函数】

```
c++
~LQ_NCNN()
```

功能：释放 ncnn 模型资源、内存空间，避免资源泄漏。

3.2 配置函数

【设置模型路径】

```
c++
void SetModelPath(const std::string& param_path, const
std::string& bin_path)
```

参数说明：

- param_path - 模型参数文件路径（.param 文件，文本格式，描述模型结构）
- bin_path - 模型权重文件路径（.bin 文件，二进制格式，存储模型权重数据）

示例：

```c++
// 设置 LQ_TinyClassifier 模型路径
ncnn.SetModelPath("/home/lq/models/LQ_TinyClassifier.param",
                  "/home/lq/models/LQ_TinyClassifier.bin");
```

【设置输入尺寸】

```c++
void SetInputSize(int width, int height)
```

参数说明:

- width - 输入图像宽度（需与模型训练时的输入尺寸一致）
- height - 输入图像高度（需与模型训练时的输入尺寸一致）

示例:

```c++
// 设置输入图像尺寸为 224x224（适配多数预训练模型）
ncnn.SetInputSize(224, 224);
```

【设置类别标签】

```c++
void SetLabels(const std::vector<std::string>& labels)
```

参数说明:

- labels - 类别名称列表，顺序需与模型训练时的类别顺序一致

示例:

```c++
// 设置类别标签（猫、狗、鸟）
std::vector<std::string> labels = {"cat", "dog", "bird"};
ncnn.SetLabels(labels);
```

【设置归一化参数】

```c++
void SetNormalize(const std::vector<float>& mean, const
```

```
std::vector<float>& norm)
```

参数说明:

- mean - 均值（RGB 三个通道，与模型训练时的归一化均值一致）
- norm - 标准差（RGB 三个通道，与模型训练时的归一化标准差一致）

示例:

```c++
// 设置 ImageNet 数据集归一化参数
std::vector<float> mean = {0.485f, 0.456f, 0.406f};
std::vector<float> norm = {0.229f, 0.224f, 0.225f};
ncnn.SetNormalize(mean, norm);
```

【设置输入输出层名称】

```c++
void SetInputName(const std::string& name)
void SetOutputName(const std::string& name)
```

参数说明:

- name - 模型输入/输出层名称（需与.param 文件中的层名称一致）

示例:

```c++
// 设置输入层名称为"data"，输出层名称为"prob"
ncnn.SetInputName("data");
ncnn.SetOutputName("prob");
```

3.3 核心函数

【初始化】

```c++
bool Init()
```

功能：加载模型并完成初始化，初始化成功后方可进行推理操作。

返回值：成功返回 true，失败返回 false（失败原因多为模型路径错误、文件损坏）。

示例:

```c++
// 初始化 ncnn 模型
if (!ncnn.Init()) {
    printf("NCNN 初始化失败!\n");
    return -1;
}
```

【推理】

```c++
std::string Infer(const cv::Mat& bgr_image)
```

功能：对输入图像进行推理，返回预测结果（类别名称或类别索引字符串）。

参数说明：

- bgr_image - OpenCV BGR 格式图像（OpenCV 默认读取格式，内部会自动转换为 RGB 格式）

返回值：预测的类别名称（已设置标签时）或类别索引字符串（未设置标签时）。

示例：

```c++
// 读取测试图像
cv::Mat image = cv::imread("test.jpg");
if (image.empty()) {
    printf("无法读取图像!\n");
    return -1;
}
// 推理并获取结果
std::string result = ncnn.Infer(image);
printf("预测结果: %s\n", result.c_str());
```

四、使用流程

使用 LQ_NCNN 类进行神经网络推理的标准流程，需遵循固定顺序，确保初始化与推理正常执行。

1. 创建 LQ_NCNN 对象
2. 设置模型路径（必需）
3. 设置输入尺寸（必需）

4. 设置类别标签（可选，根据需求配置）
5. 设置归一化参数（可选，根据模型要求配置）
6. 设置输入输出层名称（可选，默认适配 LQ_TinyClassifier）
7. 调用 Init()函数完成初始化（必需）
8. 调用 Infer()函数进行推理（必需）

五、使用示例

以下提供 4 种常见使用场景的示例代码，涵盖单一图像推理、摄像头实时推理、批量图像推理及自定义参数配置，适配不同开发需求，可直接修改路径后使用。

5.1 LQ_TinyClassifier 模型使用

LQ_NCNN 类完全适配 LQ_TinyClassifier 模型，可直接使用！
【LQ_TinyClassifier 模型训练资料】
链接: https://pan.baidu.com/s/1Yz7P73Ag9T31zfgbtxE6cA
提取码: tihd

```c++
#include "lq_ncnn.hpp"
#include <opencv2/opencv.hpp>
#include <stdio.h>

int main()
{
    // 创建 NCNN 对象
    LQ_NCNN ncnn;
    // 配置 LQ_TinyClassifier 模型路径
    ncnn.SetModelPath("/home/lq/models/LQ_TinyClassifier.param",
                      "/home/lq/models/LQ_TinyClassifier.bin");
    // 设置输入尺寸（根据模型要求）
    ncnn.SetInputSize(224, 224);
    // 设置类别标签（根据实际训练类别）
    std::vector<std::string> labels = {"cat", "dog", "bird"};
    ncnn.SetLabels(labels);
    // 初始化
    if (!ncnn.Init()) {
        printf("NCNN 初始化失败!\n");
```

```
        return -1;
    }
    // 读取测试图像
    cv::Mat image = cv::imread("test.jpg");
    if (image.empty()) {
        printf("无法读取图像!\n");
        return -1;
    }
    // 推理
    std::string result = ncnn.Infer(image);
    printf("预测结果: %s\n", result.c_str());
    return 0;
}
```

六、模型准备

ncnn 推理需使用特定格式的模型文件，需将其他框架训练的模型转换为 ncnn 支持的格式，以下详细说明模型格式、转换流程及常见预训练模型。

6.1 支持的模型格式

ncnn 推理必须使用以下两种文件，二者缺一不可：

- .param 文件：模型结构描述文件，文本格式，记录模型的网络结构、层参数等信息
- .bin 文件：模型权重数据文件，二进制格式，存储模型各层的权重、偏置等参数

6.2 模型转换流程

目前主流深度学习框架（PyTorch、TensorFlow、Caffe 等）均需转换为 ncnn 格式方可使用，常用转换流程为「PyTorch -> ONNX -> ncnn」，步骤如下：

1. 导出 ONNX 格式：将 PyTorch 模型导出为 ONNX 格式（ncnn 支持的中间格式），可通过 Python 脚本实现，示例命令：python export_onnx.py
2. 转换为 ncnn 格式：使用 ncnn 工具链中的 onnx2ncnn 工具，将 ONNX 模型转换为.param 和.bin 文件，命令：./onnx2ncnn model.onnx model.param model.bin
3. 优化模型：使用 ncnnoptimize 工具优化模型结构，提升推理速度，命令：./ncnnoptimize model.param model.bin model_opt.param model_opt.bin

6.3 常见预训练模型

以下预训练模型可在 ncnn 中使用，可在 ncnn 官方仓库获取预转换版本：

- MobileNet V2：轻量级模型，适用于嵌入式端实时推理，兼顾速度与精度
- ResNet-50：经典图像分类模型，精度较高，适用于对精度要求较高的场景
- SqueezeNet：轻量级模型，参数量少，内存占用低，适配资源有限的嵌入式设备
- ShuffleNet：高效轻量级模型，通过通道混洗提升推理速度，适用于实时场景
- EfficientNet：高效模型，精度与速度均优于传统模型，适用于复杂场景

七、常用配置参数

不同模型的输入尺寸、归一化参数、输入输出层名称存在差异，以下提供常见模型的配置参数，可直接参考使用。

7.1 不同模型的配置

【MobileNet V2】

```c++
// 输入尺寸
ncnn.SetInputSize(224, 224);
// 归一化参数
ncnn.SetNormalize({0.485f, 0.456f, 0.406f},
                  {0.229f, 0.224f, 0.225f});
```

【ResNet-50】

```c++
// 输入尺寸
ncnn.SetInputSize(224, 224);
// 归一化参数
ncnn.SetNormalize({0.485f, 0.456f, 0.406f},
                  {0.229f, 0.224f, 0.225f});
```

【SqueezeNet】

```c++
// 输入尺寸
```

```
ncnn.SetInputSize(227, 227);
// 归一化参数
ncnn.SetNormalize({0.485f, 0.456f, 0.406f},
                  {0.229f, 0.224f, 0.225f});
```

7.2 输入输出层名称

不同模型的输入输出层名称不同，需与.param 文件中的层名称一致，常见模型的层名称如下：

模型类型	输入层名	输出层名
MobileNet	data	prob
ResNet	data	fc
SqueezeNet	data	prob
自定义模型	自定义	自定义

八、使用其他模型

重要说明：LQ_NCNN 类目前完全适配 LQ_TinyClassifier 模型！如需使用其他模型（如 MobileNet、ResNet、YOLO 等），需要自行编写推理代码。

8.1 自行编写推理代码的方法

参考 LQ_NCNN 类的实现（libraries/drv/src/lq_ncnn.cpp），根据目标模型的特点，重点调整以下 3 点：

- 调整输入预处理：根据模型要求，设置正确的归一化参数、输入尺寸，完成图像格式转换（BGR→RGB）
- 设置正确的输入输出层名称：需与模型.param 文件中的层名称完全一致，否则无法正常推理
- 处理模型特定的输出格式：不同模型的输出格式不同（如分类模型输出概率、检测模型输出坐标），需对应处理输出数据

8.2 NCNN 原生 API 使用示例

若需更灵活的推理控制，可直接使用 ncnn 原生 C++ API 进行推理，示例如下：

```c++
#include <ncnn/net.h>
#include <opencv2/opencv.hpp>
#include <stdio.h>

int main()
{
    // 1. 创建 ncnn 网络对象
    ncnn::Net net;
    // 2. 加载模型参数和权重
    net.load_param("model.param");
    net.load_model("model.bin");

    // 3. 读取并预处理图像
    cv::Mat image = cv::imread("test.jpg");
    if (image.empty()) {
        printf("无法读取图像!\n");
        return -1;
    }
    // 图像尺寸调整（与模型输入尺寸一致）
    cv::resize(image, image, cv::Size(224, 224));
    // 转换为 ncnn Mat 格式（BGR→RGB）
    ncnn::Mat in = ncnn::Mat::from_pixels_resize(
        image.data, ncnn::Mat::PIXEL_BGR2RGB,
        image.cols, image.rows, 224, 224);

    // 4. 归一化处理
    const float mean_vals[3] = {0.485f*255.f, 0.456f*255.f,
0.406f*255.f};
    const float norm_vals[3] = {1/0.229f/255.f, 1/0.224f/255.f,
1/0.225f/255.f};
    in.substract_mean_normalize(mean_vals, norm_vals);

    // 5. 执行推理
    ncnn::Extractor ex = net.create_extractor();
    ex.input("data", in);  // 设置输入层
    ncnn::Mat out;
    ex.extract("prob", out);  // 获取输出层结果

    // 6. 处理输出结果（以分类模型为例，获取概率最大的类别）
    float max_prob = 0;
```

```
    int max_index = 0;
    for (int i = 0; i < out.w; i++) {
        if (out[i] > max_prob) {
            max_prob = out[i];
            max_index = i;
        }
    }
    printf("预测类别索引: %d, 概率: %.4f\n", max_index, max_prob);

    // 7. 释放资源
    net.clear();
    return 0;
}
```

九、注意事项

- 1. 初始化顺序
  - 必须先设置模型路径、输入尺寸等核心参数，再调用 Init()函数初始化
  - 禁止未初始化就调用 Infer()函数，否则会导致程序崩溃
- 2. 图像格式
  - 输入图像必须为 OpenCV BGR 格式（OpenCV imread()函数默认格式）
  - LQ_NCNN 类内部会自动将 BGR 格式转换为 RGB 格式，无需手动转换
- 3. 输入尺寸
  - 设置的输入尺寸必须与模型训练时的输入尺寸一致，否则会导致推理结果错误
  - 常见输入尺寸：224x224（多数分类模型）、256x256、299x299（复杂模型）
- 4. 归一化参数
  - 归一化参数必须与模型训练时的设置一致，否则会严重影响推理精度
  - 若模型训练时未进行归一化，无需调用 SetNormalize()函数
- 5. 性能优化
  - 使用 ncnnoptimize 工具优化模型，可提升推理速度
  - 使用 ncnn2int8 工具将模型量化为 INT8 精度，可大幅提升速度、降低内存占用（精度略有损失）
  - 实时推理时，可适当降低输入分辨率，平衡速度与精度

- 6. 错误处理
  - 调用 Init()函数后，务必判断返回值，处理初始化失败的场景
  - 读取图像后，需判断图像是否为空，避免因图像路径错误导致推理异常
- 7. 内存管理
  - 大尺寸图像需先缩放至模型输入尺寸再进行推理，避免内存溢出
  - 批量处理图像时，需及时释放图像资源，避免内存占用过高
  - 程序结束前，确保释放 ncnn 模型资源（析构函数会自动释放，无需手动操作）

十、常见问题

Q1: 模型加载失败（Init()返回 false）？

A1: 检查以下几点：

- 模型文件路径是否正确，确保.param 和.bin 文件路径与 SetModelPath()设置一致
- 模型文件是否存在且可读，可通过 ls 命令查看文件权限
- 模型文件格式是否正确，确保是 ncnn 支持的.param 和.bin 格式，而非 ONNX、PyTorch 格式

Q2: 推理结果不正确、偏差较大？

A2: 检查以下几点：

- 输入尺寸是否与模型训练时一致，尺寸不匹配会导致特征提取错误
- 归一化参数是否正确，需与模型训练时的均值、标准差完全一致
- 类别标签顺序是否与模型训练时一致，标签顺序错误会导致预测类别错误
- 输入图像格式是否为 BGR，若手动转换为 RGB 会导致颜色通道错误

Q3: 推理速度慢，无法满足实时需求？

A3: 尝试以下优化方法：

- 使用 ncnnoptimize 工具优化模型结构，减少冗余计算
- 使用 ncnn2int8 工具将模型量化为 INT8 精度，提升推理速度
- 降低输入图像分辨率（如从 224x224 降至 160x160）
- 减少推理频率，避免连续高频调用 Infer()函数

Q4: 程序运行时出现内存不足错误？

A4: 建议采取以下措施：

- 降低输入图像尺寸，减少内存占用
- 批量处理图像时，及时释放 cv::Mat 对象，避免内存堆积
- 使用 INT8 量化模型，降低模型内存占用

Q5: 如何查看模型的输入输出层名称?

A5: 直接打开模型的.param 文件（文本格式），查看文件开头的层定义，通常第一个输入层为"input"或"data"，最后一个输出层为"output"、"prob"或"fc"。

Q6: 如何使用 YOLO 等目标检测模型?

A6: LQ_NCNN 类暂未适配目标检测模型，需自行编写推理代码：

- 参考 ncnn 官方 YOLO 系列示例，适配龙芯板卡
- 重点处理模型输出的目标坐标、置信度、类别等信息，完成目标框绘制

十一、参考资源

- ncnn 官方仓库：https://github.com/Tencent/ncnn
- ncnn 官方文档：https://ncnn.readthedocs.io/
- 本库 ncnn 封装相关文件：
  - 头文件：libraries/drv/inc/lq_ncnn.hpp
  - 实现文件：libraries/drv/src/lq_ncnn.cpp
  - 静态库：libraries/drv/src/liblqncnn.a
- 示例代码：
  - ncnn 基础示例：example/src/07_lq_ncnn_demo.cpp
- ncnn 工具链位置：tools/LQ_Dep_libs/ncnn_install/bin/
- LQ_TinyClassifier 模型训练资料：
  - 链接: https://pan.baidu.com/s/1Yz7P73Ag9T31zfgbtxE6cA
  - 提取码: tihd